Abstract Factory

Season: 1 · Creational · Category: GoF · Level: Intermediate · Build with Waqar

Today's Hook

- Today: Abstract Factory as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Abstract Factory structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Abstract Factory
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/creational/abstractfactory/UiFactory.java
package com.buildwithwaqar.patterns.gof.creational.abstractfactory;

interface Button { String paint(); }
interface Checkbox { String paint(); }

record LightButton() implements Button {
    public String paint() { return "light-button"; }
}
record DarkButton() implements Button {
    public String paint() { return "dark-button"; }
}
record LightCheckbox() implements Checkbox {
    public String paint() { return "light-checkbox"; }
}
record DarkCheckbox() implements Checkbox {
    public String paint() { return "dark-checkbox"; }
}

/** Family of related products without specifying concrete classes. */
public interface UiFactory {
    Button button();
    Checkbox checkbox();
```

Implementation (2)

```
    static UiFactory of(Theme theme) {
        return switch (theme) {
            case LIGHT -> new LightFactory();
            case DARK -> new DarkFactory();
        };
    }
}

enum Theme { LIGHT, DARK }

record LightFactory() implements UiFactory {
    public Button button() { return new LightButton(); }
    public Checkbox checkbox() { return new LightCheckbox(); }
}

record DarkFactory() implements UiFactory {
    public Button button() { return new DarkButton(); }
    public Checkbox checkbox() { return new DarkCheckbox(); }
}

record Dialog(UiFactory factory) {
    String render() {
        return factory.button().paint() + "+" + factory.checkbox().paint();
    }
```

Implementation (3)

```
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.abstractfactory;

public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Abstract Factory walkthrough ===");
        System.out.println("light=" + new Dialog(UiFactory.of(Theme.LIGHT)).render());
        System.out.println("dark=" + new Dialog(UiFactory.of(Theme.DARK)).render());
    }
}
```

Walkthrough Output

```
=== Abstract Factory walkthrough ===
light=light-button+light-checkbox
dark=dark-button+dark-checkbox
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Abstract Factory
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Abstract Factory → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.